

ELECTRONICS ENGINEERING

MICROPROCESSORS LABORATORY

2023-2024 FALL

EXPERIMENT-4

Group Members

Name	Fouad	Büşra	Ayşe Beril
Surname	Aladhami	Ülker	Çevik
Student ID	210102002808	200102002011	2101020002020

1 PROBLEM 1 3

1.1 FLOW CHART 3
1.2 CODE 3

2 PROBLEM 2 5

2.1 FLOW CHART 6
2.2 SCHEMATIC 7
2.3 CODE 7

3 PROBLEM 3 10

3.1 FLOW CHART 11
3.2 SCHEMATIC 12
3.3 CODE 12

4 CONCLUSION 21

5 REFERENCES 21

1 PROBLEM 1

The aim of this project is creating the functions of transmitting and receiving data.

When we observe the code, first GPIOA, USART2 module clocks and PA2, PA3 pins are enabled from RCC. Then PA2 and PA3 pins settled as alternate function for the USART2 module. CR1 control register choose from datasheet at enabling receive and transmit from USART2 module. TDR and RDR registers used at data transmit and receive processes respectively. Baud rate settled to 9600 bps assuming clock is running at 16Mhz. Value chosen according to the formula of baud rate and EISA RS-232 communication Standard.

Baud Rate = APBxCLK / USARTx

Some of the standard baud rates: 300, 600, 1200, 2400, 4800, 9600, 14400, 19200, 38400, 57600, 115200, 230400, 460800.

1.1 FLOW CHART

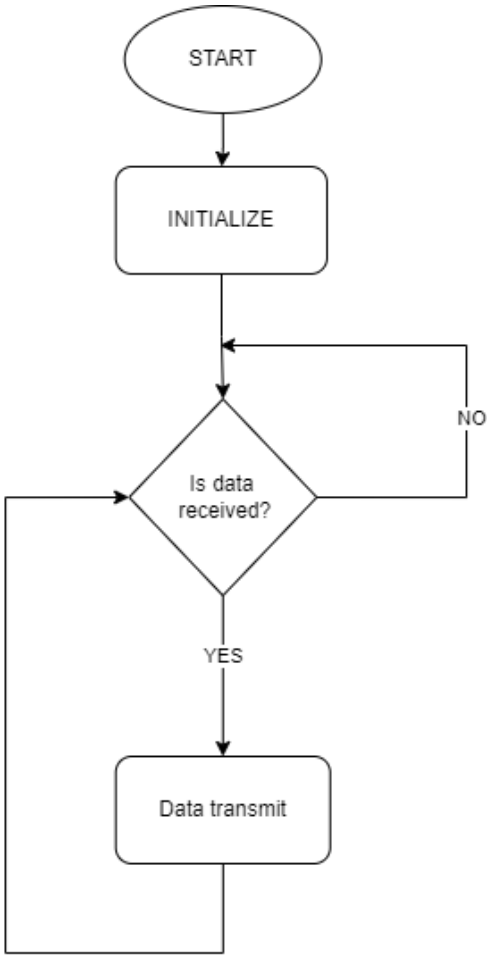


Figure 1: Flow chart of problem1.

1.2 CODE

```c
#include "stm32g0xx.h"
#include "stm32g031xx.h"
void RCC_Init(void);
void GPIOA_Init(void);
void USART2_Init(void);
void printChar(uint8_t c);
int _print(int f, char *ptr, int len);
void print(char *s);
uint8_t uart_rx(void);
void uart_tx(uint8_t c);

int main(void)
{
	RCC_Init();
	GPIOA_Init();
	USART2_Init();

	while (1)
	{
		uart_tx(uart_rx());
	}
}

void RCC_Init(void)
{
	// enable GPIOA and USART2 module clocks from RCC
	RCC->IOPENR |= RCC_IOPENR_GPIOAEN;
	RCC->APBENR1 |= RCC_APBENR1_USART2EN;
}
void GPIOA_Init(void)
{
	// enable and setup pins PA2 and PA3 as alternate function for USART2 module
	GPIOA->MODER &= ~(GPIO_MODER_MODE2 | GPIO_MODER_MODE3);
	GPIOA->MODER |= (GPIO_MODER_MODE2_1 | GPIO_MODER_MODE3_1);
	GPIOA->AFR[0] &= ~(GPIO_AFRL_AFSEL2 | GPIO_AFRL_AFSEL3);
	GPIOA->AFR[0] |= (GPIO_AFRL_AFSEL2_0 | GPIO_AFRL_AFSEL3_0);
}
void USART2_Init(void)
{
	// enable receive and transmit from USART2 module
	USART2->CR1 |= (USART_CR1_RE | USART_CR1_TE);
	// set baud rate to 9600 bps assuming clock is running at 16Mhz
	//Baud Rate = APBxCLK / USARTx->BRR
	USART2->BRR = 0x683;//1667
	// enable uart module
	USART2->CR1 |= USART_CR1_UE;
}
void printChar(uint8_t c)
{
	// write c to the transmit data register
	USART2->TDR = c;
	// wait until transmit is complete
	while (!(USART2->ISR & USART_ISR_TC))
		;
}
int _print(int f, char *ptr, int len)
{
	(void)f; // don't do anthing with f
	// call printChar within a for loop len times
	for (int i = 0; i < len; i++)
	{
		printChar(ptr[i]);
	}
	return len; // return length
}
void print(char *s)
```

```
{
        // count number of characters in s string until a null byte comes `\0`
        int length = 0;
        while (s[length] != '\0')
        {
                length++;
        }
        _print(0, s, length);
}
uint8_t uart_rx(void)
{
        // wait until receive is complete
        while (!(USART2->ISR & USART_ISR_RXNE_RXFNE))
                ;
        // return the received data
        return USART2->RDR;
}
void uart_tx(uint8_t c)
{
        // write c to the transmit data register
        USART2->TDR = c;
        // wait until transmit is complete
        while (!(USART2->ISR & USART_ISR_TC))
                ;
}
```

2 PROBLEM 2

Pulse Width Modulation is a technique used to encode information in the duration of a pulse signal. By changing the duty cycle of the PWM signal LED brightness can be changed.

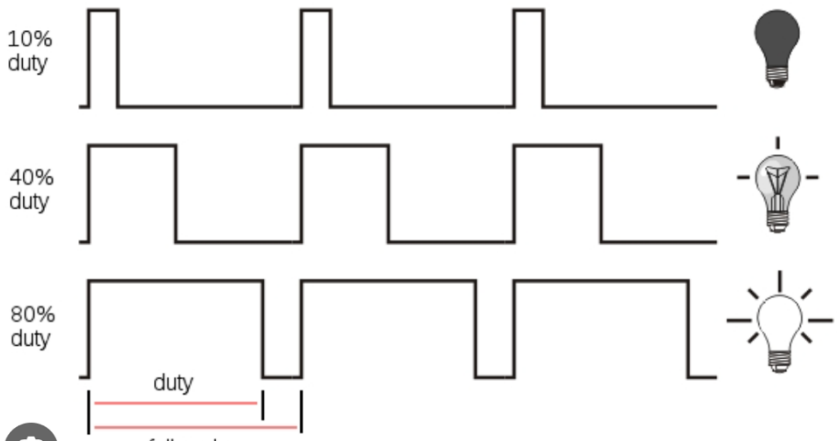


Figure 2: Duty cycle led brightness relationship.

PWM involves rapidly toggling a digital signal between high and low states. The frequency of the PWM signal can calculated by the above formula:

$$PWM\ frequency = \frac{APB\ clock\ frequency}{ARR * (PSC + 1)}$$

To set the PWM frequency to 1 Khz, because the clock is default 16Mhz, PSC register value is set as 15, and ARR register value as 1000. To set the duty cycle the CCR register value can changed. The duty cycle percentage can be calculated by the above formula:

$$Duty\ Cycle = \frac{CCR}{ARR} * 100\ \%$$

So as an example, to set duty cycle to 40% CCR value must be equal to 400.
TIM2 is used to generate the PWM signal, and the interrupt is activated. In the interrupt the duty cycle of the PWM signal is updated. By using the Systick timer a counter is increased and decreased. The counter is set as PWM duty cycle in the TIM2 interrupt. So, every 1ms the counter increase by 1, this is equal to 0.1% of duty cycle.

2.1 FLOW CHART

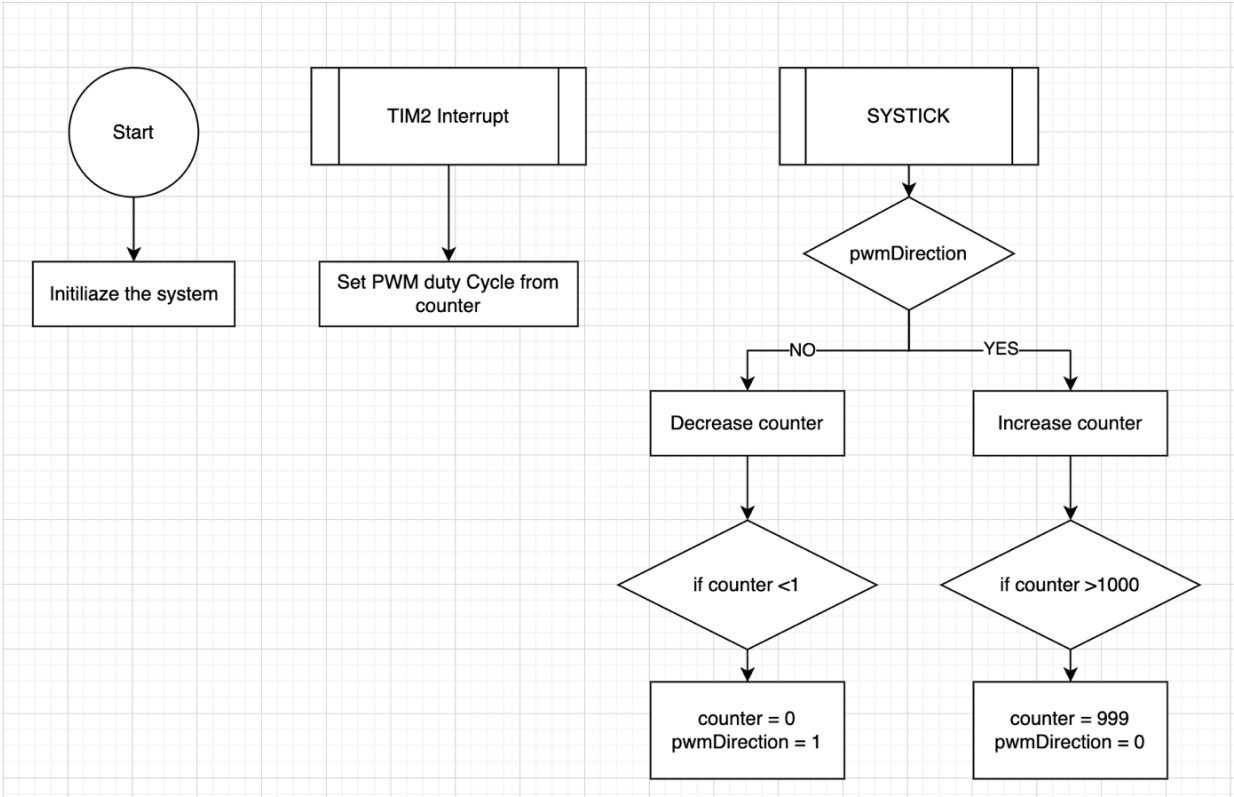


Figure 3: Flowchart of problem 2

2.2 SCHEMATIC

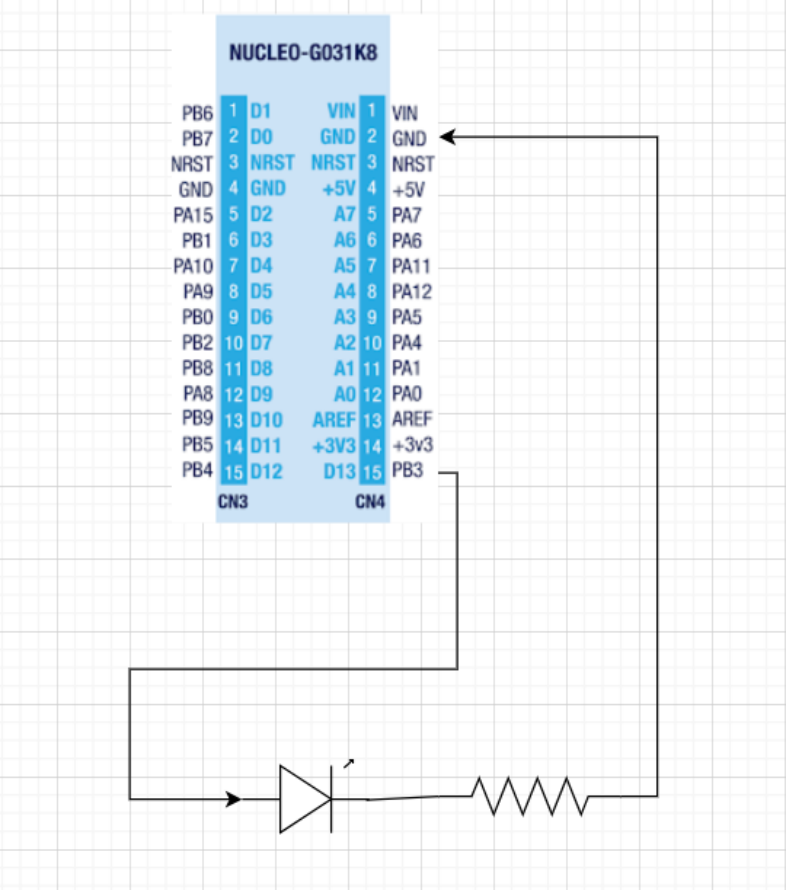


Figure 4: Schematic of pwm led circuit

2.3 Code

```
/*
 * main.c
 */
#include "main.h"

void Init_Systick(uint32_t tick);
void increase_tick(void);
void TIM2_IRQHandler(void);
void SetDutyCycle(uint16_t dutyCycle);
void StartPWM(void);
void StopPWM(void);
void SysTick_Handler(void);
void init_pwm2(void);

// variables
uint64_t tick;
uint8_t seconds;
uint16_t counter = 0;
uint8_t pwmDirection = 1; // 1 for increase, 0 for decrease

// defines
#define SYSTICK_FREQ 16000 // Khz
int main(void)
```

```
{

    Init_Systick(SYSTICK_FREQ);
    init_pwm2();
    SetDutyCycle(0);
    StartPWM();

    while (1)
    {
    }

    return 0;
}

void TIM2_IRQHandler(void)
{
    // update duty
       SetDutyCycle(counter);
    // Clear update status register
    TIM2->SR &= ~(1U << 0);
}

/**
 * Setup PWM output for
 * TIM2 CH2 on PB3
 */
void init_pwm2(void)
{

    // Setup GPIO
    //

    // Enable GPIOB clock
    RCC->IOPENR |= (1U << 1);
    // Enable TIM2 clock
    RCC->APBENR1 |= RCC_APBENR1_TIM2EN;

    // Set alternate function to 2
    // 3 comes from PB3
    GPIOB->AFR[0] |= (2U << 4 * 3);
    // Select AF from Moder
    GPIOB->MODER &= ~(3U << 2 * 3);
    GPIOB->MODER |= (2U << 2 * 3);

    // zero out the control register just in case
    TIM2->CR1 = 0;

    // Select PWM Mode 1
    TIM2->CCMR1 |= (6U << 12);
    // Preload Enable
    TIM2->CCMR1 |= TIM_CCMR1_OC2PE;

    // Capture compare ch2 enable
    TIM2->CCER |= TIM_CCER_CC2E;

    // zero out counter
    TIM2->CNT = 0;
    // 1 ms interrupt
    //CLOCK/((PSC+1)* ARR)
    TIM2->PSC = 15;
    TIM2->ARR = 1000;

    // zero out duty
    TIM2->CCR2 = 0;

    // Update interrupt enable
```

```
    TIM2->DIER |= (1 << 0);

    // TIM1 Enable
    // TIM2->CR1 |= TIM_CR1_CEN;

    NVIC_SetPriority(TIM2_IRQn, 1);
    NVIC_EnableIRQ(TIM2_IRQn);
}
void StartPWM(void)
{
    // Enable the timer
    TIM2->CR1 |= TIM_CR1_CEN;
}

void StopPWM(void)
{
    // Disable the timer
    TIM2->CR1 &= ~TIM_CR1_CEN;
}
void SetDutyCycle(uint16_t dutyCycle)
{
    // Ensure duty cycle is within bounds

    // Set the duty cycle by updating the CCR2 register
    TIM2->CCR2 = dutyCycle;
    TIM2->CCR1 = dutyCycle;
}
void increase_tick(void)
{
    tick++;
}
void Init_Systick(uint32_t tick)
{
    SysTick->LOAD = tick; // Count down from 999 to 0
    SysTick->VAL = 0;     // Clear current value
    SysTick->CTRL = 0x7;  // Enable Systick, exception,and use processor clock
}
/**
 * @brief This function handles System tick timer.
 */
void SysTick_Handler(void)
{
    increase_tick();
    if (pwmDirection)
    {
        counter++;
        if (counter > 1000)
        {
            counter = 999;
            pwmDirection = 0;
        }
    }
    else
    {
        counter--;
        if (counter < 1)
        {
            counter = 0;
            pwmDirection = 1;
        }
    }

}
```

3 PROBLEM 3

The columns of the keypad were set as output and rows as input. Columns were connected to EXTI and keypad keys were controlled. First the column was determined from the incoming interrupt, then the rows were controlled and the key pressed was determined. Significant numbers entered were stored until "# came. Significant values between 0 and 100 affected the duty cycle, while other meaningless values and signs gave a warning. PWM was controlled with TIM2, UART was controlled with TIM3.

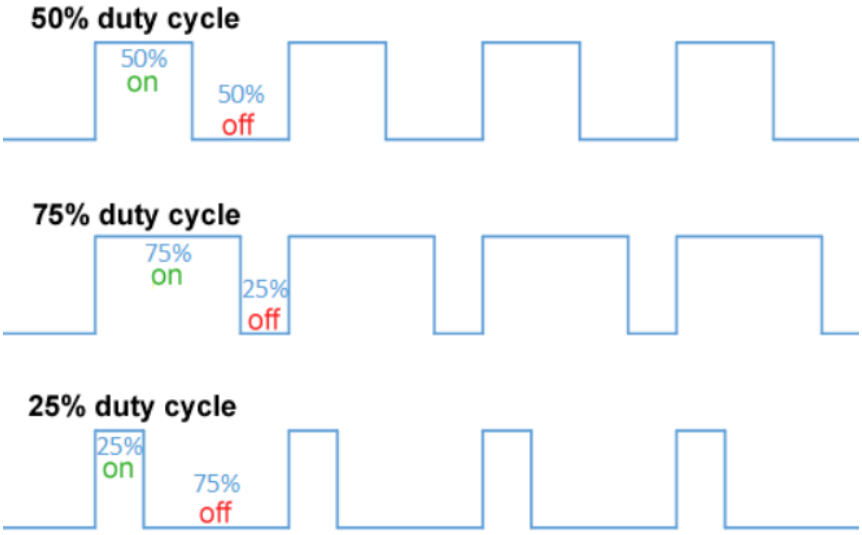


Figure 5: Duty cycle of pwm signal

The internal structure of 4X4 KEYPAD MODULE is shown below.

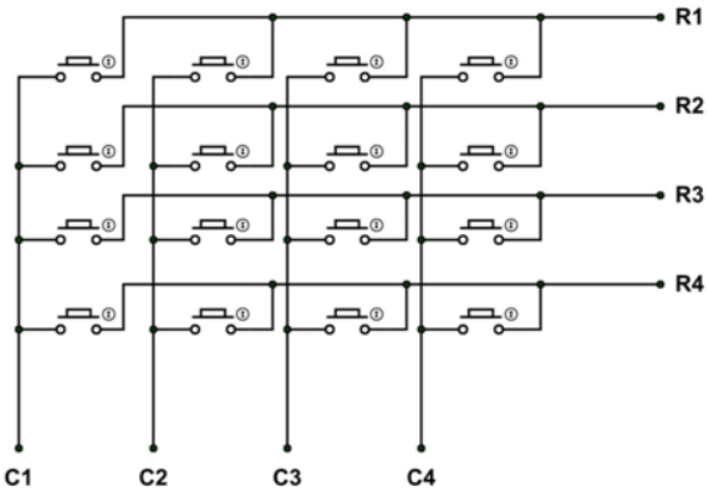


Figure 5: Keypad circuit model

3.1 FLOW CHART

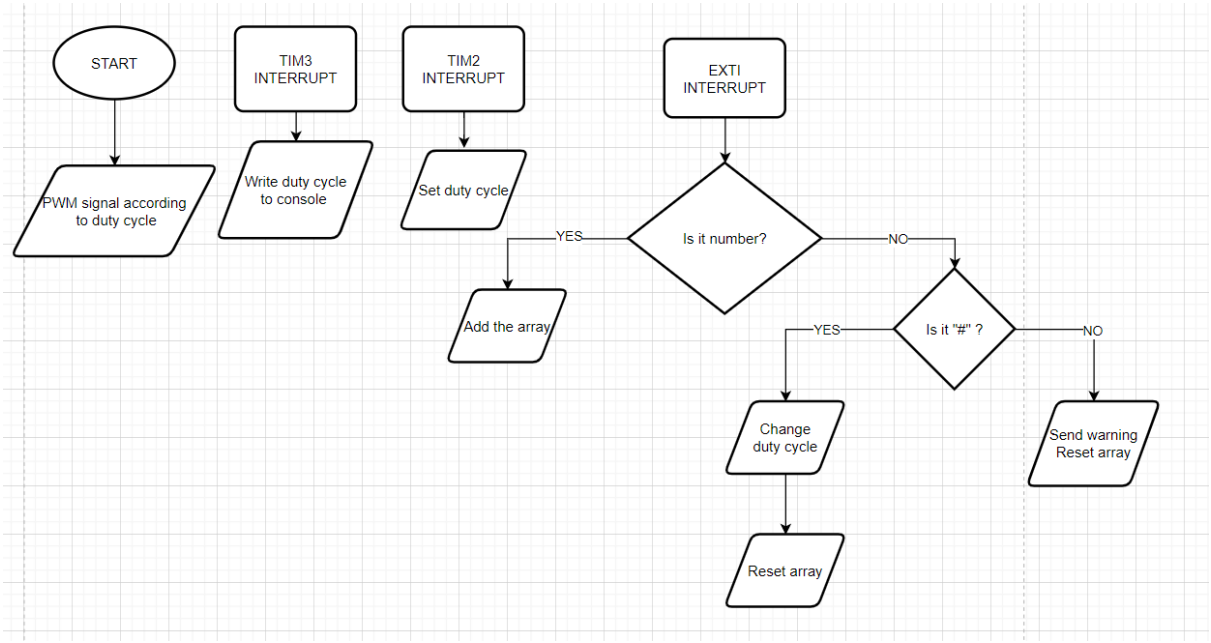


Figure 6: Problem 3 flowchart

3.2 SCHEMATIC

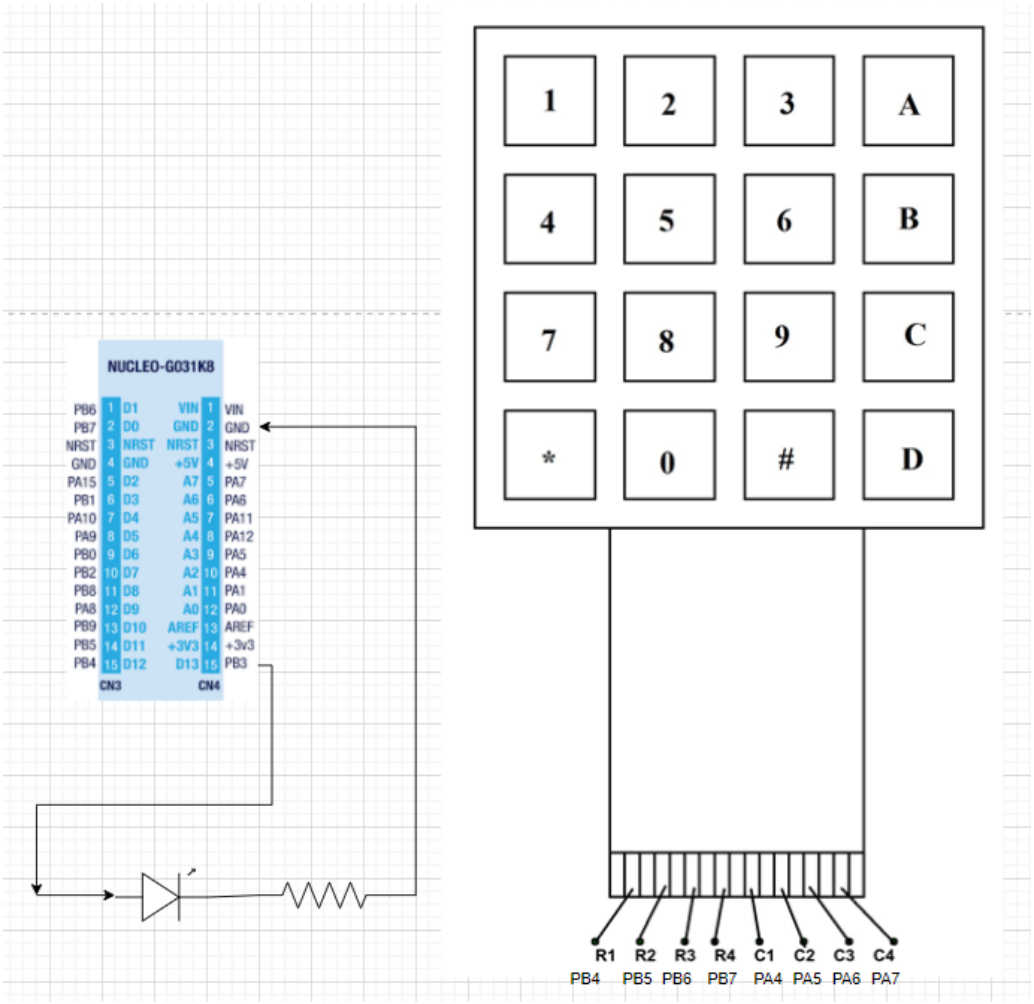


Figure 6: Problem 3 Schematic

3.3 CODE

```
#include "stm32g0xx.h"
#include <math.h>
#include "main.h"
#include <stdio.h>

void clearRowsKeypad(void);
void setRowsKeypad(void);
void storenumber(int digit);
void RCC_Init(void);
void GPIOA_Init(void);
void GPIOB_Init(void);
void EXTI_Init(void);
void DelayMs(uint32_t ms);
void EXTI4_15_IRQHandler(void);
void USART2_Init(void);
void printChar(uint8_t c);
int _print(int f, char *ptr, int len);
void print(char *s);
void printDutyCycle(uint8_t dutyCyclePercentage);
void TIM3_IRQHandler(void);
```

```c
void Init_TIM3(void);

uint32_t number = 0; // number of stored digits
uint32_t count;               // duty cycle rate
uint32_t ust;
uint8_t b;
uint32_t numbers[3]; // store digit

// pwm variables and prototpes
void Init_Systick(uint32_t tick);
void increase_tick(void);
void TIM2_IRQHandler(void);
void SetDutyCycle(uint16_t dutyCycle);
void StartPWM(void);
void StopPWM(void);
void SysTick_Handler(void);
void init_pwm2(void);

// variables
uint64_t tick;
uint8_t seconds;
uint16_t pwm_CCR_value = 0;
uint8_t pwmDirection = 1; // 1 for increase, 0 for decrease

// defines
#define SYSTICK_FREQ 16000 // Khz

int main(void)
{
	RCC_Init();
	GPIOB_Init(); /*Set up PB4,PB5,PB6,PB7 as output row(output=01)*/
	GPIOA_Init(); /*Set up PA4,PA5,PA6,PA7 as input column(input=00)*/
	Init_Systick(SYSTICK_FREQ);
	init_pwm2();
	Init_TIM3();
	EXTI_Init();
	setRowsKeypad(); /*Set all rows*/
	SetDutyCycle(0);
	StartPWM();
	USART2_Init();

	while (1)
	{
	}
	return 0;
}

void RCC_Init(void)
{
	RCC->IOPENR |= (1U << 0); /*Enable GPIOA clock*/
	RCC->IOPENR |= (1U << 1); /*Enable GPIOB clock*/
	// Enable clock for SYSCFG
	RCC->APBENR2 |= RCC_APBENR2_SYSCFGEN;
	// Enables GPIOA peripheral
	RCC->AHBENR |= 1;
	// Enable usart2 clock
	RCC->APBENR1 |= RCC_APBENR1_USART2EN;
	// Enable TIM2 clock
	RCC->APBENR1 |= RCC_APBENR1_TIM2EN;
	RCC->APBENR1 |= RCC_APBENR1_TIM3EN;
}

void GPIOB_Init(void)
{
	/*Set up PB4,PB5,PB6,PB7 as output row(output=01)*/
	GPIOB->MODER &= ~(3U << 8);
```

```
        GPIOB->MODER |= (1U << 8); /*for PB4*/

        GPIOB->MODER &= ~(3U << 10);
        GPIOB->MODER |= (1U << 10); /*for PB5*/

        GPIOB->MODER &= ~(3U << 12);
        GPIOB->MODER |= (1U << 12); /*for PB6*/

        GPIOB->MODER &= ~(3U << 14);
        GPIOB->MODER |= (1U << 14); /*for PB7*/

        // Select AF from Moder
        GPIOB->MODER &= ~(3U << 2 * 3);
        GPIOB->MODER |= (2U << 2 * 3);

        // Set alternate function to 2
        // 3 comes from PB3
        GPIOB->AFR[0] |= (2U << 4 * 3);
}

void GPIOA_Init(void)
{
        /*Set up PA4,PA5,PA6,PA7 as input column(input=00)*/
        GPIOA->MODER &= ~(3U << 8); /*for PA4*/
        GPIOA->PUPDR |= (2U << 8);   /*Pull down mode*/

        GPIOA->MODER &= ~(3U << 10); /*for PA5*/
        GPIOA->PUPDR |= (2U << 10);   /*Pull down mode*/

        GPIOA->MODER &= ~(3U << 12); /*for PA6*/
        GPIOA->PUPDR |= (2U << 12);   /*Pull down mode*/

        GPIOA->MODER &= ~(3U << 14); /*for PA7*/
        GPIOA->PUPDR |= (2U << 14);   /*Pull down mode*/

        // enable and setup pins PA2 and PA3 as alternate function for USART2 module
        GPIOA->MODER &= ~(GPIO_MODER_MODE2 | GPIO_MODER_MODE3);
        GPIOA->MODER |= (GPIO_MODER_MODE2_1 | GPIO_MODER_MODE3_1);
        GPIOA->AFR[0] &= ~(GPIO_AFRL_AFSEL2 | GPIO_AFRL_AFSEL3);
        GPIOA->AFR[0] |= (GPIO_AFRL_AFSEL2_0 | GPIO_AFRL_AFSEL3_0);
}

void EXTI_Init(void)
{
        /*Set up interrupt for input*/
        EXTI->EXTICR[2] &= ~EXTI_EXTICR2_EXTI4_Msk; /*PA4*/
        EXTI->EXTICR[2] &= ~EXTI_EXTICR2_EXTI5_Msk; /*PA5*/
        EXTI->EXTICR[2] &= ~EXTI_EXTICR2_EXTI6_Msk; /*PA6*/
        EXTI->EXTICR[2] &= ~EXTI_EXTICR2_EXTI7_Msk; /*PA7*/

        /*Rising edge*/
        EXTI->RTSR1 |= EXTI_RTSR1_RT4;
        EXTI->FTSR1 &= ~EXTI_FTSR1_FT4;
        EXTI->RTSR1 |= EXTI_RTSR1_RT5;
        EXTI->FTSR1 &= ~EXTI_FTSR1_FT5;
        EXTI->RTSR1 |= EXTI_RTSR1_RT6;
        EXTI->FTSR1 &= ~EXTI_FTSR1_FT6;
        EXTI->RTSR1 |= EXTI_RTSR1_RT7;
        EXTI->FTSR1 &= ~EXTI_FTSR1_FT7;

        /*MASK*/
        EXTI->IMR1 |= EXTI_IMR1_IM4;
        EXTI->IMR1 |= EXTI_IMR1_IM5;
        EXTI->IMR1 |= EXTI_IMR1_IM6;
        EXTI->IMR1 |= EXTI_IMR1_IM7;
```

```
        /*NVIC*/

        NVIC_SetPriority(EXTI4_15_IRQn, 3);
        NVIC_EnableIRQ(EXTI4_15_IRQn);
}

void DelayMs(uint32_t ms)
{
        for (uint32_t i = 0; i < ms * 1000; i++)
        {
                // Hiçbir şey yapma - sadece beklet
        }
}

void storenumber(int digit)
{
        DelayMs(300);
        if (digit >= 0 && digit <= 9 && number != 3)
        {
                numbers[number] = digit;
                number = number + 1;
        }
        else if (number == 3)
        {
                print("You enter a number biger than 100, please retry again\n");
                number = 0;
                numbers[0] = 0;
                numbers[1] = 0;
                numbers[2] = 0;
        }
        // if hashtag pressed
        else if (digit == 10 && number != 0)
        {
                int i;

                for (i = 0; i < number; i++)
                {
                        ust = pow(10, number - i - 1);
                        count = count + numbers[i] * ust;
                }
                if (count > 100)
                {
                        print("You enter a number biger than 100, please retry again\n");
                }
                else
                {
                        pwm_CCR_value = count * 10;
                        // print duty cycle
                        printDutyCycle(pwm_CCR_value / 10);
                }
                count = 0;
                number = 0;
                numbers[0] = 0;
                numbers[1] = 0;
                numbers[2] = 0;
        }
        else if (digit == 11  )
        {
                count = 0;
                number = 0;
                numbers[0] = 0;
                numbers[1] = 0;
                numbers[2] = 0;
                print("Undefined input charackter, please retry again\n");
        }
        else if (digit == 10  )
```

```
		{
			count = 0;
			number = 0;
			numbers[0] = 0;
			numbers[1] = 0;
			numbers[2] = 0;
			print("Undefined input charackter, please retry again\n");
		}
}
void printDutyCycle(uint8_t dutyCyclePercentage)
{
	// print the duty cycle using print function
	char str[4] = "/0";
	sprintf(str, "%d", dutyCyclePercentage);
	print("Duty Cycle: ");
	print(str);
	print("\n");
}

void EXTI4_15_IRQHandler(void)
{

	if ((EXTI->RPR1 >> 4) & 1)
	{ // Interrupt from PA4
		clearRowsKeypad();

		GPIOB->ODR ^= (1U << 4); // PB4
		if ((GPIOA->IDR >> 4) & 1)
		{
			storenumber(1);
		}
		GPIOB->ODR ^= (1U << 4); // PB4

		GPIOB->ODR ^= (1U << 5); // PB5
		if ((GPIOA->IDR >> 4) & 1)
		{
			storenumber(4);
		}
		GPIOB->ODR ^= (1U << 5); // PB5

		GPIOB->ODR ^= (1U << 6); // PB6
		if ((GPIOA->IDR >> 4) & 1)
		{
			storenumber(7);
		}
		GPIOB->ODR ^= (1U << 6); // PB6

		GPIOB->ODR ^= (1U << 7); // PB7
		if ((GPIOA->IDR >> 4) & 1)
		{
			storenumber(11);
		}
		GPIOB->ODR ^= (1U << 7); // PB7

		EXTI->RPR1 |= (1U << 4); // clear interrupt flag
		setRowsKeypad();
	}

	if ((EXTI->RPR1 >> 5) & 1)
	{ // Interrupt from PA5

		clearRowsKeypad();
		GPIOB->ODR ^= (1U << 4); // PB4
		if ((GPIOA->IDR >> 5) & 1)
		{
			storenumber(2);
```

```
		}
		GPIOB->ODR ^= (1U << 4); // PB4

		GPIOB->ODR ^= (1U << 5); // PB5
		if ((GPIOA->IDR >> 5) & 1)
		{
			storenumber(5);
		}
		GPIOB->ODR ^= (1U << 5); // PB5

		GPIOB->ODR ^= (1U << 6); // PB6
		if ((GPIOA->IDR >> 5) & 1)
		{
			storenumber(8);
		}
		GPIOB->ODR ^= (1U << 6); // PB6

		GPIOB->ODR ^= (1U << 7); // PB7
		if ((GPIOA->IDR >> 5) & 1)
		{
			storenumber(0);
		}
		GPIOB->ODR ^= (1U << 7); // PB7
		EXTI->RPR1 |= (1U << 5); // clear interrupt flag
		setRowsKeypad();
	}

	if ((EXTI->RPR1 >> 6) & 1)
	{ // Interrupt from PA6
		clearRowsKeypad();
		GPIOB->ODR ^= (1U << 4); // PB4
		if ((GPIOA->IDR >> 6) & 1)
		{
			storenumber(3);
		}
		GPIOB->ODR ^= (1U << 4); // PB4

		GPIOB->ODR ^= (1U << 5); // PB5
		if ((GPIOA->IDR >> 6) & 1)
		{
			storenumber(6);
		}
		GPIOB->ODR ^= (1U << 5); // PB5

		GPIOB->ODR ^= (1U << 6); // PB6
		if ((GPIOA->IDR >> 6) & 1)
		{
			storenumber(9);
		}
		GPIOB->ODR ^= (1U << 6); // PB6

		GPIOB->ODR ^= (1U << 7); // PB7
		if ((GPIOA->IDR >> 6) & 1)
		{
			storenumber(10);
		}
		GPIOB->ODR ^= (1U << 7); // PB7

		EXTI->RPR1 |= (1U << 6); // clear interrupt flag
		setRowsKeypad();
	}
	if ((EXTI->RPR1 >> 7) & 1)
	{ // Interrupt from PA7
		clearRowsKeypad();
		GPIOB->ODR ^= (1U << 4); // PB4
		if ((GPIOA->IDR >> 7) & 1)
```

```
                {
                        storenumber(11);
                }
                GPIOB->ODR ^= (1U << 4); // PB4

                GPIOB->ODR ^= (1U << 5); // PB5
                if ((GPIOA->IDR >> 7) & 1)
                {
                        storenumber(11);
                }
                GPIOB->ODR ^= (1U << 5); // PB5

                GPIOB->ODR ^= (1U << 6); // PB6
                if ((GPIOA->IDR >> 7) & 1)
                {
                        storenumber(11);
                }
                GPIOB->ODR ^= (1U << 6); // PB6

                GPIOB->ODR ^= (1U << 7); // PB7
                if ((GPIOA->IDR >> 7) & 1)
                {
                        storenumber(11);
                }
                GPIOB->ODR ^= (1U << 7); // PB7

                EXTI->RPR1 |= (1U << 7); // clear interrupt flag
                setRowsKeypad();
        }
}

void clearRowsKeypad(void)
{
        /*Clearing the rows*/
        GPIOB->ODR &= ~(1U << 4); /*PB4*/
        GPIOB->ODR &= ~(1U << 5); /*PB5*/
        GPIOB->ODR &= ~(1U << 6); /*PB6*/
        GPIOB->ODR &= ~(1U << 7); /*PB7*/
}

void setRowsKeypad(void)
{
        /*Setting rows*/
        GPIOB->ODR |= (1U << 4); /*PB4*/
        GPIOB->ODR |= (1U << 5); /*PB5*/
        GPIOB->ODR |= (1U << 6); /*PB6*/
        GPIOB->ODR |= (1U << 7); /*PB7*/
}
void TIM2_IRQHandler(void)
{
        // update duty
        SetDutyCycle(pwm_CCR_value);
        // Clear update status register
        TIM2->SR &= ~(1U << 0);
}
void TIM3_IRQHandler(void)
{
        // update duty
        // print duty cycle
        printDutyCycle(pwm_CCR_value / 10);
        // Clear update status register
        TIM3->SR &= ~(1U << 0);
}

/**
 * Setup PWM output for
```

```
 * TIM2 CH2 on PB3
 */
void init_pwm2(void)
{
	// zero out the control register just in case
	TIM2->CR1 = 0;

	// Select PWM Mode 1
	TIM2->CCMR1 |= (6U << 12);
	// Preload Enable
	TIM2->CCMR1 |= TIM_CCMR1_OC2PE;

	// Capture compare ch2 enable
	TIM2->CCER |= TIM_CCER_CC2E;

	// zero out counter
	TIM2->CNT = 0;
	// 1 ms interrupt
	// CLOCK/((PSC+1)* ARR)
	TIM2->PSC = 15;
	TIM2->ARR = 1000;

	// zero out duty
	TIM2->CCR2 = 0;

	// Update interrupt enable
	TIM2->DIER |= (1 << 0);

	// TIM1 Enable
	// TIM2->CR1 |= TIM_CR1_CEN;

	NVIC_SetPriority(TIM2_IRQn, 1);
	NVIC_EnableIRQ(TIM2_IRQn);
}
// Make TIM 3 for 2 second interrupt
void Init_TIM3(void)
{
	// zero out the control register just in case
	TIM3->CR1 = 0;

	// zero out counter
	TIM3->CNT = 0;
	// 1 ms interrupt
	// CLOCK/((PSC+1)* ARR)
	TIM3->PSC = 15999;
	TIM3->ARR = 2000;

	// Update interrupt enable
	TIM3->DIER |= (1 << 0);

	// TIM1 Enable
	TIM3->CR1 |= TIM_CR1_CEN;

	NVIC_SetPriority(TIM3_IRQn, 2);
	NVIC_EnableIRQ(TIM3_IRQn);
}
void StartPWM(void)
{
	// Enable the timer
	TIM2->CR1 |= TIM_CR1_CEN;
}

void StopPWM(void)
{
	// Disable the timer
	TIM2->CR1 &= ~TIM_CR1_CEN;
```

```
}
void SetDutyCycle(uint16_t dutyCycle)
{
	// Ensure duty cycle is within bounds

	// Set the duty cycle by updating the CCR2 register
	TIM2->CCR2 = dutyCycle;
	TIM2->CCR1 = dutyCycle;
}
void increase_tick(void)
{
	tick++;
}
void Init_Systick(uint32_t tick)
{
	SysTick->LOAD = tick; // Count down from 999 to 0
	SysTick->VAL = 0;	// Clear current value
	SysTick->CTRL = 0x7;  // Enable Systick, exception,and use processor clock
}
/**
 * @brief This function handles System tick timer.
 */
void SysTick_Handler(void)
{
	increase_tick();
}
void USART2_Init(void)
{
	// enable receive and transmit from USART2 module
	USART2->CR1 |= (USART_CR1_RE | USART_CR1_TE);
	// set baud rate to 9600 bps assuming clock is running at 16Mhz
	// Baud Rate = APBxCLK / USARTx->BRR
	USART2->BRR = 0x683; // 1667
	// enable uart module
	USART2->CR1 |= USART_CR1_UE;
}
void printChar(uint8_t c)
{
	// write c to the transmit data register
	USART2->TDR = c;
	// wait until transmit is complete
	while (!(USART2->ISR & USART_ISR_TC))
		;
}
int _print(int f, char *ptr, int len)
{
	(void)f; // don't do anthing with f
	// call printChar within a for loop len times
	for (int i = 0; i < len; i++)
	{
		printChar(ptr[i]);
	}
	return len; // return length
}
void print(char *s)
{
	// count number of characters in s string until a null byte comes `\0`
	int length = 0;
	while (s[length] != '\0')
	{
		length++;
	}
	_print(0, s, length);
}
```

4 CONCLUSION

The change of significant values entered from the keypad was observed on the LED by changing the duty cycle and the logic of PWM duty cycle operation was understood. In the meantime, Keypad control was provided with External Interrupts and the response of the microcontroller to the entry is provided using UART, as an example the duty cycle value is sent to the computer every 2 seconds, and when an incorrect entry is entered a warning message is sent. So using UART the microcontroller can communicate with the external word.

5 REFERENCES

1.rm0444-stm32g0x1-advanced-armbased-32bit-mcus-stmicroelectronics

2.https://components101.com/misc/4x4-keypad-module-pinout-configuration-features-datasheet